Force-Directed Scheduling for High-Level Synthesis: Mechanism, Comparative Context, and Critical Evaluation

Md Jubair Salehin Razin
1230281
Department of Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany
md-jubair-salehin.razin@stud.hshl.de

***Abstract*—High-level synthesis turns a behavioral description of a digital system into a register-transfer-level design, and one of its central tasks is scheduling: deciding in which clock cycle each operation of a data-flow graph executes. This paper gives a self-contained account of force-directed scheduling, the heuristic introduced by Paulin and Knight for the time-constrained problem of meeting a fixed latency with the fewest functional units. We explain the full mechanism: as-soon-as-possible and as-late-as-possible analyses produce a time frame and a mobility for every operation; treating each operation as uniformly distributed over its frame yields a distribution graph that estimates the expected demand on each resource type per cycle; and a spring-like force, combining a self term with predecessor and successor terms, scores every candidate placement so that the schedule which flattens demand is built one decision at a time. We support the account with a compact C++ implementation whose output is used as the worked example, and we show how the resulting schedule maps onto an RTL netlist of VHDL entities. We then place the method in context against unconstrained bounding schedules, greedy list scheduling, and exact integer linear programming, summarizing the tradeoffs in a single comparison. Finally we evaluate force-directed scheduling critically, separating its concrete strengths from its hidden assumptions, its scalability, the embedded cases where it does and does not fit, and one extension we propose. We argue that the method is a deliberate middle ground that gains a global view without the cost of exact search, but that per-decision recomputation, the absence of backtracking, and its reliance on uniform distributions and data-independent delays bound its use to small and medium datapaths.**

***Index Terms*—high-level synthesis, scheduling, force-directed scheduling, resource minimization, data-flow graph, architectural synthesis**

I. Introduction

High-level synthesis (HLS) compiles a behavioral specification of a digital system into a register-transfer-level (RTL) implementation—a netlist of RTL components such as functional units, registers and a controller, each of which can be realized as a VHDL entity—raising design productivity by letting the designer reason about algorithms rather than gates [2], [3]. Within HLS, *architectural synthesis* turns a data-flow or sequencing graph into a datapath and a controller through two coupled tasks: *scheduling*, which assigns each operation to a control step (clock cycle), and *binding*, which maps operations and values onto physical resources such as functional units and registers [2]. Scheduling fixes the temporal concurrency of the design and therefore largely determines how many resources the binding step will need.

Two dual formulations dominate. In *resource-constrained* scheduling the number of functional units is fixed and latency is minimized; in *time-constrained* scheduling the latency (number of control steps) is fixed and the resource count is minimized. The time-constrained case is the natural model for fixed-throughput signal-processing datapaths, where the sample period pins the latency and the designer wants the cheapest hardware that meets it.

Simple bounding schedules—*as soon as possible* (ASAP) and *as late as possible* (ALAP)—minimize latency while ignoring resources [2]. *List scheduling* handles the resource-constrained problem with a fast greedy priority rule, and *integer linear programming* (ILP) solves either problem exactly at exponential worst-case cost [2]. Force-directed scheduling (FDS), introduced by Paulin and Knight [1], [4], targets the time-constrained problem with a constructive heuristic that balances operation concurrency *globally* rather than greedily.

This paper consolidates a technical account of FDS (Section II), positions it against the alternatives (Section III), and evaluates it critically (Section IV). Throughout we separate textbook facts and reported results (cited) from our own interpretation, critique, and explicitly flagged speculation.

II. The Force-Directed Scheduling Algorithm

A. Problem statement and assumptions

Let the behavior be a data-flow graph $G(V, E)$ in which each vertex $v_i \in V$ is an operation and each edge $(v_i, v_j) \in E$ is a data dependency requiring v_i to finish before v_j starts. FDS solves the following: given a latency bound λ (the number of available control steps), assign a start step to every operation

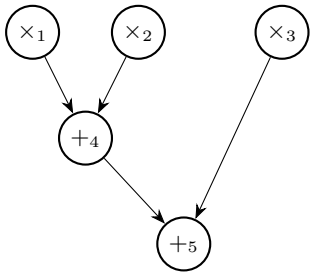

Fig. 1. Running example: a data-flow graph with three multiplications ($\times_1, \times_2, \times_3$) and two additions ($+_4, +_5$). The critical path is $\times_1 \to +_4 \to +_5$ (length 3); the latency bound is $\lambda = 4$.

TABLE I
TIME FRAMES FOR THE EXAMPLE OF FIG. 1 ($\lambda = 4$).

Operation	Type	t_S	t_L	μ
v_1	$\times$	1	2	1
v_2	$\times$	1	2	1
v_3	$\times$	1	3	2
v_4	$+$	2	3	1
v_5	$+$	3	4	1

so that all dependencies hold, the schedule fits in λ steps, and the number of functional units of each type is as small as possible [1].

The classical formulation assumes (i) *data-independent delays*—operation latencies are known constants and there are no schedule-altering conditional branches; (ii) each operation occupies a *single control step* and one functional unit of its type, with multi-cycle operations, operation chaining, and pipelined units admitted through documented extensions [1]; and (iii) a fixed, feasible λ that is at least the critical-path length. Under these assumptions, minimizing the per-type peak concurrency over the schedule is a sound proxy for minimizing functional units, and this is exactly what FDS attacks.

B. Time frames and mobility

FDS first computes the two bounding schedules. The ASAP schedule $t_S(v_i)$ places each operation at the earliest step its dependencies permit (the longest path from the sources); the ALAP schedule $t_L(v_i)$ places it at the latest step that λ permits [2]. The interval $[\,t_S(v_i),\, t_L(v_i)\,]$ is the *time frame* of v_i, and its width

$$\mu(v_i) = t_L(v_i) - t_S(v_i) \tag{1}$$

is the *mobility* (slack). Operations with $\mu = 0$ lie on the critical path and cannot move; operations with $\mu > 0$ can be shifted to relieve resource pressure. Table I lists the time frames for the running example of Fig. 1 ($\lambda = 4$).

C. Distribution graphs

FDS models an as-yet-unscheduled operation as *uniformly distributed* over its time frame: the probability that v_i occupies step l is

$$p_i(l) = \begin{cases} \dfrac{1}{\mu(v_i)+1}, & t_S(v_i) \le l \le t_L(v_i), \\ 0, & \text{otherwise.} \end{cases} \tag{2}$$

For each resource type k, the *distribution graph* (DG) sums these probabilities over all operations V_k of that type,

$$\mathrm{DG}_k(l) = \sum_{v_i \in V_k} p_i(l), \tag{3}$$

giving the *expected* number of type-k operations in step l. In the example, v_3 (a multiplier with $\mu = 2$) contributes $1/3$ to each of s_1, s_2, s_3, while v_1 and v_2 ($\mu = 1$) each contribute $1/2$ to s_1 and s_2. Under the ASAP schedule all three multiplications would start in s_1, demanding three multipliers; the distribution graph instead spreads the expected demand, peaking at 1.33 in s_1 and s_2 (Fig. 2, "initial").

D. Forces

The distribution graph is read as a system of springs. A spring's restoring force is its stiffness times its displacement; here the stiffness is the local DG value (high where a step is already crowded) and the displacement is the change in an operation's probability caused by a tentative scheduling decision [1]. Scheduling v_i into step j replaces its distribution by a unit impulse at j; the resulting *self-force* is

$$F^{\mathrm{self}}(v_i, j) = \sum_{l=t_S(v_i)}^{t_L(v_i)} \mathrm{DG}_k(l)\,\big(\delta_{jl} - p_i(l)\big), \tag{4}$$

where $\delta_{jl} = 1$ if $l = j$ and 0 otherwise. A positive self-force means the assignment pushes probability into already-busy steps (undesirable); a negative self-force means it moves work toward idle steps (desirable). For the example, assigning v_3 to the crowded s_1 gives $F^{\mathrm{self}}(v_3, s_1) = +0.33$, whereas assigning it to the near-empty s_3 gives $F^{\mathrm{self}}(v_3, s_3) = -0.67$.

Because dependencies couple operations, fixing v_i may shrink the time frames of its predecessors and successors, changing *their* probabilities too. Each induced change contributes a *predecessor force* or *successor force*, and the quantity FDS actually minimizes is the total

$$F(v_i, j) = F^{\mathrm{self}}(v_i, j) + \sum_{\text{pred/succ}} F^{\mathrm{ind}}. \tag{5}$$

In the example, placing v_3 at s_3 also forces its successor v_5 to s_4, and the accompanying successor force is likewise negative, reinforcing the choice.

E. The scheduling loop

FDS is constructive. At each iteration it (1) recomputes time frames and distribution graphs, (2) evaluates the total force (5) for every unscheduled operation in every step of its frame, and (3) irrevocably commits the single operation–step pair of *lowest* force. Committing one operation tightens neighboring frames; the loop repeats until every operation is scheduled. In the example the first commitment is $v_3 \to s_3$; the remaining

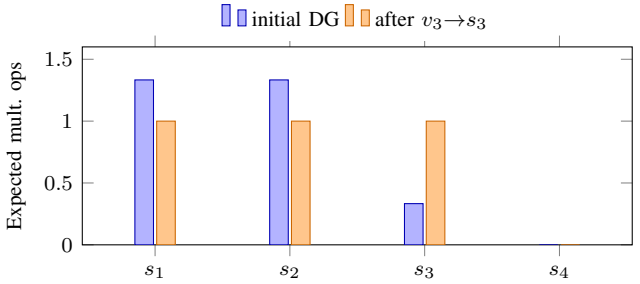

Fig. 2. Multiplier distribution graph for the example. The initial graph is peaked in s_1–s_2; committing $v_3 \to s_3$ flattens it to one expected operation per step, which a single multiplier can serve.

commitments place $v_2 \to s_1$, $v_1 \to s_2$, $v_4 \to s_3$ and $v_5 \to s_4$, so the multiplier demand becomes one operation per step (Fig. 2, "after") and a single multiplier serves the schedule where ASAP would have required three.

F. Runtime and memory

The recurring cost is force evaluation, repeated after every one of the $O(|V|)$ commitments; each pass touches the operations across their frames and propagates frame changes to neighbors. The distribution graphs require $O(R \cdot \lambda)$ storage for R resource types plus $O(|V|)$ for the frames—modest. Time is the bottleneck: the per-decision recomputation makes FDS roughly quadratic-to-cubic in the number of operations, markedly costlier than list scheduling yet far below the worst-case-exponential ILP [1], [2].

G. Reference implementation

To make the mechanism concrete and to check the worked example, FDS was implemented in portable C++ as a single self-contained file using only the standard library. The data-flow graph is stored as operations carrying their type and their predecessor/successor lists; ASAP and ALAP are longest-path passes over a topological order; the distribution graph is accumulated from the current frames through (3); and the loop of Section II-E commits, at each iteration, the operation–step pair of lowest force. The *main feature* is the force routine of Listing 1: it pins a candidate operation at a candidate step, recomputes all time frames, and sums the distribution-weighted change in probability over every operation whose frame moves. Because the pin is propagated through the ASAP/ALAP passes, the self-force and the predecessor/successor force of (5) emerge from this one loop rather than being coded separately.

```
double force(const Problem& p,
             const vector<int>& fixed_step,
             int op, int step) {
  Frames before = compute_frames(p, fixed_step);
  if (!before.feasible ||
      step < before.asap[op] ||
      step > before.alap[op])
    return numeric_limits<double>::infinity();

  // DG from the CURRENT frames (the spring stiffness)
  auto dg = distribution_graph(p, before);

  // pin op at step, then recompute every frame
  vector<int> trial = fixed_step;  trial[op] = step;
  Frames after = compute_frames(p, trial);
  if (!after.feasible)
    return numeric_limits<double>::infinity();

  double total_force = 0.0;
  for (int u = 0; u < p.n(); ++u) {
    if (fixed_step[u] != -1 && u != op) continue;
    int type = p.ops[u].type;
    for (int s = 1; s <= p.latency; ++s) {
      double oldP = probability(before, u, s);
      double newP = probability(after,  u, s);
      total_force += dg[type][s] * (newP - oldP);
    }
  }
  return total_force;   // self + pred/succ force together
}
```

Listing 1. Main feature: total-force evaluation of a candidate pair.

Run on the graph of Fig. 1, the program reproduces the time frames of Table I and the distribution graph of Fig. 2 exactly. Its first commitment is $v_3 \to s_3$, which carries the most negative total force, -0.917 (the self-force of -0.67 together with a negative successor force); the remaining commitments are $v_2 \to s_1$, $v_1 \to s_2$, $v_4 \to s_3$ and $v_5 \to s_4$. The multiplier peak falls from three under the ASAP schedule to one and the adder peak stays at one, confirming the hand analysis.

H. From schedule to an RTL netlist

The product of HLS is not the schedule itself but the RTL netlist the schedule and the resource allocation imply: a set of functional units, the registers that carry values across control-step boundaries, and a controller. For the scheduled example this netlist is *one multiplier* and *one adder/ALU*, both shared across control steps, the registers holding the intermediate results v_1–v_4, and a four-state finite-state controller— one state per control step—that in each step drives the operand multiplexers of the shared units and latches their outputs. Each functional unit is a separate VHDL entity; Listing 2 shows the multiplier entity. The controller activates the units exactly as the schedule prescribes: s_1 computes v_2 and s_2 computes v_1 on the shared multiplier, s_3 computes v_3 on the multiplier and v_4 on the adder in parallel, and s_4 computes v_5 on the adder. Against the ASAP baseline, which would instantiate three multipliers, the force-directed schedule yields a netlist with a single multiplier at the same latency.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity mul_unit is
  port( a : in  signed(31 downto 0);
        b : in  signed(31 downto 0);
        y : out signed(63 downto 0) );
end entity;

architecture rtl of mul_unit is
begin
  y <= a * b;
end architecture;
```

Listing 2. Shared multiplier as a VHDL entity.

TABLE II
SCHEDULING METHODS FOR HIGH-LEVEL SYNTHESIS, COMPARED ALONG THREE TRADEOFF AXES.

Method	Problem solved	Optimality	Scope of each decision	Relative cost
ASAP / ALAP	Unconstrained, minimize latency	Exact bounds; no resource view	Per operation	Very low (linear)
List scheduling	Resource-constrained, minimize latency	Heuristic, greedy	Local; one control step at a time	Low (near-linear)
ILP	Either problem, exact	Optimal	Global, exhaustive search	Very high (worst-case exponential)
Force-directed (FDS)	Time-constrained, minimize resources	Heuristic, near-optimal	Global balancing with look-ahead	Moderate (quadratic–cubic)

III. FDS IN THE SCHEDULING LANDSCAPE

A. Unconstrained bounds: ASAP and ALAP

Both run in time linear in the graph and minimize latency while disregarding resources; they are not solutions to the resource-minimization problem but the *inputs* FDS consumes to build time frames [2].

B. Resource-constrained greedy: list scheduling

Given a fixed number of units, list scheduling walks the control steps in order and, at each step, dispatches ready operations to free units in the order of a priority—commonly urgency, i.e. inverse mobility [2]. It is fast and is the workhorse of production tools, but its decisions are *local*: it never looks beyond the current step, so it can build avoidable concurrency peaks.

C. Exact: integer linear programming

ILP encodes a binary variable for every legal operation–step pair, with dependency, resource, and latency constraints and an objective of minimal resources or latency [2]. It returns provably optimal schedules, but its worst-case cost is exponential, confining it to small kernels or to calibrating heuristics.

D. Tradeoffs

Table II arranges the four methods along three axes that, in our reading, fully separate them: *heuristic vs. optimal* (FDS and list are heuristic, ILP optimal); *which quantity is fixed* (FDS fixes latency and minimizes resources, list fixes resources and minimizes latency); and *global balancing vs. greedy locality* (FDS weighs every step through the distribution graph before each move, while list commits per step). FDS is thus the natural choice when latency is given, resources are precious, and the instance is too large for ILP yet large enough that greedy peaks hurt.

E. On the sparseness of related work

For FDS itself the primary literature is essentially a single thread—Paulin and Knight's conference and journal papers [4], [1]—while the comparison methods are standard textbook material [2], [3]. We therefore keep the reference set deliberately small: a longer citation list would pad rather than inform, because the conceptual neighbors of FDS are precisely the four methods compared above.

IV. CRITICAL EVALUATION

A. Strengths

The force is a *principled, instance-specific* cost: instead of an ad-hoc priority, every candidate move is scored by its measured effect on global balance, which is why FDS reaches near-optimal resource counts on time-constrained datapaths where greedy list scheduling leaves peaks [1]. The distribution-graph machinery is also *extensible*— multi-cycle operations, operation chaining, mutually exclusive operations, and pipelined units are folded in by adjusting how probabilities enter (3) and (4) [1]—so one framework covers a family of problems rather than a single case.

B. Limitations and hidden assumptions

Three concerns stand out. First, *cost*: the force must be recomputed after each commitment, the source of the quadratic-to-cubic runtime; on large graphs this dominates. Second, and in our assessment more fundamental, FDS is *greedy at the meta-level*: although each move is globally informed, the algorithm commits one operation per iteration and never backtracks, so a locally minimal force can steer it into a globally suboptimal schedule. Third, the *uniform-distribution* premise of (2) is a modeling convenience, not a fact about optimal placements; where the true distribution is skewed by dependency structure, the distribution graph mis-estimates pressure and the forces are correspondingly biased. Underlying all of this is the *data-independent-delay* assumption: once control flow alters timing, the notion of a fixed per-step expected occupancy weakens.

C. Scalability

Memory scales benignly ($O(R\lambda + |V|)$), but runtime does not scale linearly, so FDS is comfortable on the tens-to-low-hundreds of operations typical of arithmetic kernels and is strained well before the operation counts of large, control-dominated blocks.

D. Application fit

FDS fits *fixed-rate DSP datapaths*—FIR/IIR filters, transforms, and the differential-equation kind of kernel used in the original study [1]— where the sample period fixes λ and the goal is to minimize multipliers and adders; this is its home ground. It fits *poorly* where behavior is *control-dominated* with data-dependent branching and variable latency

(the assumptions break), and where the design is large enough that per-decision force recomputation is prohibitive. For a hard-real-time block whose requirement is a guaranteed worst case under a fixed resource budget, the resource-constrained formulation—list scheduling, or ILP for small cases—is a better match than time-constrained FDS.

E. An extension and an open question

A practical extension, partially explored in the scheduling literature, is to use the force only as the *priority* inside an otherwise greedy list scheduler ("force-directed list scheduling"), trading some global accuracy for a single linear pass; alternatively, FDS can be followed by a local-search or annealing refinement that reintroduces the backtracking the base method lacks. As an open question we *conjecture* (speculation) that the weakest link is the uniform-probability model of (2): a dependency-aware—or even data-driven—probability estimate could yield distribution graphs that track true contention more faithfully and, with it, better schedules at the same asymptotic cost.

V. Conclusion

Force-directed scheduling is a deliberate middle ground in time-constrained high-level synthesis: it buys the global view that greedy list scheduling lacks without paying the exponential price of exact optimization, and it does so through a single, intuitive idea—balance the expected per-step resource demand by treating the distribution graph as a system of springs. A compact C++ implementation reproduced the worked example, and the resulting schedule mapped directly onto an RTL netlist of one multiplier, one adder and a four-state controller. The method's strengths (a principled, extensible cost; near-optimal results on small-to-medium datapaths) and its limits (per-decision recomputation, no backtracking, and reliance on uniform distributions and data-independent delays) both follow directly from that central idea. It remains a reference point for scheduling research and a clear lens through which to understand the resource–latency tradeoff.

Acknowledgment

The author thanks Prof. Dr. Achim Rettberg and M.Sc. Ali Alhalabi for their supervision of this seminar.

Declaration on the Use of AI Tools

In preparing this paper, a large language model (Claude, Anthropic) was used to assist with structuring, drafting, and LaTeX formatting, and with describing the author's own C++ and VHDL. All technical statements were checked against the cited sources, and the C++ implementation was executed by the author to verify every numerical result. The comparative positioning and the critical evaluation—the choice of tradeoff axes, the strengths and limitations asserted, the application-fit judgement, and the proposed extension—are the author's own. Fact, interpretation, critique, and speculation are kept separate throughout the text, and the author is solely responsible for the content.

References

[1] P. G. Paulin and J. P. Knight, "Force-directed scheduling for the behavioral synthesis of ASIC's," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 8, no. 6, pp. 661–679, Jun. 1989.

[2] G. De Micheli, *Synthesis and Optimization of Digital Circuits*. New York, NY, USA: McGraw-Hill, 1994.

[3] J. Teich, *Digitale Hardware/Software-Systeme: Synthese und Optimierung*. Berlin, Germany: Springer-Verlag, 1997.

[4] P. G. Paulin and J. P. Knight, "Force-directed scheduling in automatic data path synthesis," in *Proc. 24th ACM/IEEE Design Automation Conf. (DAC)*, Miami Beach, FL, USA, 1987, pp. 195–202.